

Migrate the Data to the New Data Architecture

Estimated time: 30 minutes

Learning objectives

- Apply data migration techniques to transfer data to the new architecture.
- Design appropriate schemas and migration strategies that align with the EDA standards during the transition.

About SN Labs and Cloud IDE

Skills Network (SN) Labs offer a fully functional environment - Cloud IDE - for course and project-related labs. It runs on Theia, an open-source integrated development environment (IDE) compatible with desktop and cloud platforms.

With this Cloud IDE, you can access Theia and MySQL, both running in a Docker container. It has all the essential tools needed to complete your lab efficiently.

Consideration to work in the lab environment

This lab environment does not save your session data. Each time you connect, a new environment is created.

To prevent data loss, plan to complete the lab in a single session.

Introduction

In this lab, you will execute data migration activities per the Enterprise Data Architecture (EDA) blueprint developed in the project's earlier phases. The goal is to mimic real-world like data integration scenarios by operating in a staging environment. This enables secure testing and experimentation before implementing changes in the production environment.

You will execute the following migrations:

- Transfer data from a Relational Database Management System (RDBMS) to a NoSQL database.
- Transfer data from a NoSQL database to a Relational Database Management System (RDBMS).

These exercises will allow you to appreciate the intricacies of working across different database technologies and ensure your data migration strategies align with the architectural standards you've established.

Let's get started!

Exercise 1: Migrating data from RDBMS to NoSQL database

TrendyThreads initially stored all product catalog information in a traditional Relational Database Management System (RDBMS). However, due to the flexibility and scalability offered by schemaless databases, a NoSQL database has been chosen as the new platform for managing product catalog data. In this exercise, you will migrate data from the MySQL database to the MongoDB database.

Notes: This mongoimport command reference [file](#) will be helpful in finishing this exercise.

Step 0: Initialize the source RDBMS database

1. **Start the MySQL server:** Ensure the MySQL service is up and running on your system.
2. **Download the database initialization script:** Run the command below to download the database initializer script.

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/zAjsXWY5_32IwexYkhzrAw/initialize-mysqldb.py
```

3. **Initialize the MySQL database:** Run the commands below to initialize the MySQL server.

```
python3 -m pip install mysql-connector-python pymongo
```

```
python3 initialize-mysqldb.py
```

Step 1: Perform the data migration

Migrate the contents of the product_catalog database from the MySQL server to the product_catalog database in MongoDB.

Step 2: Run the post migration validation script

1. **Download the validation script:** Run the command below to download the lab check script

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/CS4WZfaShihm741Uk4JYeg/mysql2mongo-lab-check.py
```

2. **Execute the validation script to capture the migration status:** Run the commands below to generate the migration report.

```
python3 mysql2mongo-lab-check.py
```

3. Save the generated output file:

- The above script will generate a text file named **rdbms2nosql.txt**.
- Download and save this file to your local machine.
- This file is required for final project submission.

Exercise 2: Migrating data from NoSQL to RDBMS

FashionMart originally stored structured product returns data in a schemaless NoSQL database. However, to better manage structured and relational data, the organization has decided to migrate this information to a Relational Database Management System (RDBMS).

In this exercise, you will migrate product returns data from a MongoDB database to a MySQL database while designing the appropriate schema for structured storage.

Step 0: Initialize the source MongoDB database

1. **Start the MongoDB server:** Make sure the MongoDB service is running.
2. **Download the database initialization script:** Run the command below to download the database initializer script.

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/sEYK9WTXN6K6Ho3aKhCmpw/initialize-mongodb.py
```

3. **Initialize the MongoDB database:** Run the commands below to initialize the MySQL server.

```
python3 -m pip install mysql-connector-python pymongo
```

```
python3 initialize-mongodb.py
```

Step 1: Perform the data migration

Migrate the data from the product_returns database on the MongoDB server to the table in the product_returns database on the MySQL server. Ensure the data is correctly transformed and mapped during the migration process.

Step 2: Run the post migration validation script

1. **Download the validation script:** Run the command below to download the lab check script.

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/WpWvD4SopTBr3K7yIEWDWA/mongo2mysql-lab-check.py
```

2. **Execute the validation script to capture the migration status:** Run the command below to generate the migration report.

```
python3 mongo2mysql-lab-check.py
```

3. **Save the generated output file:**

- The above script will generate a text file named **nosql2rdbms.txt**.
- Download and save this file to your local machine.
- This file is required for final project submission.

Summary

Congratulations on completing this Lab: Migrate the Data to the New Data Architecture. In this lab, you executed a critical part of FutureMart's data transformation by implementing the migration strategy defined in your EDA blueprint. You began by migrating product catalog data from TrendyThreads' traditional RDBMS to a NoSQL database, enabling a more flexible and scalable solution for dynamic, schemaless data. Next, you transitioned structured data from FashionMart's NoSQL system into a relational database, ensuring consistency, data integrity, and better support for structured queries.

Author(s)

Ramesh Sannareddy